

Nokta-İçinde-Poligon (Point-in-Polygon) Tespiti

Paralel Ray Casting Algoritmasının Gerçekleştirilmesi

Üniversite Bursa Teknik Üniversitesi Ders: Paralel Programlama Dersi Bölüm: Bilgisayar Mühendisliği Bölümü Öğrenci: Mehmet Özay (22360859062)

Proje Bağlantıları

- GitHub Reposu: <https://github.com/ozay-mehmet/Point-In-Polygon>
- Proje Gösterim Videosu: <https://youtu.be/VHv8HkVzL3w>

1. Proje Özeti

Bu projede, bilgisayar grafiklerinin ve geometrik hesaplamaların temel problemlerinden biri olan **Point-in-Polygon (PIP)** probleminin çözümü için "Ray Casting" algoritmasının Java ortamında paralel olarak gerçekleştirilmesi amaçlanmıştır.

Parallelleştirme işlemi, modern Java'nın güçlü eşzamanlılık (concurrency) araçlarından biri olan **ForkJoinPool** yapısı kullanılarak uygulanmıştır. Convex (dışbükey) ve concave (içbükey) poligonlar üzerinde yapılan kapsamlı testlerde, sıralı (sequential) ve paralel çözümlerin %100 tutarlı sonuçlar ürettiği kanıtlanmıştır. Yapılan performans ölçümleri, özellikle veri miktarı arttıkça paralel yaklaşımın ciddi hızlanma (speedup) sağladığını göstermektedir.

Anahtar Kelimeler: Paralel Programlama, Ray Casting, Point-in-Polygon, ForkJoinPool, Java Concurrency, Speedup.

2. Problem Tanımı

Büyük veri kümeleri üzerinde gerçekleştirilen hesaplamalarda, işlemlerin çok çekirdekli işlemcilere dağıtılması çalışma sürelerini dramatik ölçüde azaltır.

Problem: Sisteme verilen bir çokgenin (poligon) koordinatları ve binlerce test noktası kullanılarak, her bir test noktasının poligonun **içinde mi** yoksa **dışında mı** olduğunun tespit edilmesidir.

Neden Parallelleştirme? Her bir noktanın tespiti, diğer noktalardan tamamen bağımsızdır. Bu durum, problemi *Embarrassingly Parallel* (Kolayca Parallelleştirilebilir) sınıfına sokar. Projenin ana motivasyonu, bu durumu fırsata çevirerek performans artışını analiz etmektir.

3. Ray Casting Algoritması

Ray Casting, PIP problemi için endüstri standardı haline gelmiş yöntemlerden biridir.

Çalışma Mantığı:

- Test noktasından pozitif S_x ekseni yönünde sanal yatay bir ışın (ray) gönderilir.

2. Bu işinin poligonun kenarlarıyla kaç kez kesiştiği sayılır.

[!TIP] **Temel Kural:**

- Kesişim sayısı **TEK** ise: Nokta poligonun **İÇİNDEDİR**.
- Kesişim sayısı **ÇİFT** ise: Nokta poligonun **DIŞINDADIR**.

Bu yöntem algoritmik olarak $O(n)$ zaman karmaşıklığına sahiptir (n = poligonun kenar sayısı).

⚡ 4. Mimari ve Paralel Çözüm Tasarımı

Projede iş yükü dağıtımı için verilerin sistemdeki işlemci çekirdekleri (CPU Cores) arasında paylaşılması stratejisi benimsenmiştir.

Thread-Safe Tasarım (Race Condition Çözümü)

Veri yarışı (Race Condition) problemleri projede **lock, mutex veya synchronized anahtar kelimeleri kullanılmadan** çözülmüştür.

- Poligon verisi **salt okunur (read-only)** olarak paylaşılır.
- Her iş parçacığı (thread), kendisine ayrılan bellek bölgesine (sonuç dizisinin spesifik indekslerine) yazar. Bu sayede bellek çakışması engellenir.

Temel Sınıf Yapısı

Sınıf Adı	Açıklama ve Görev
Point	2D uzaydaki X, Y koordinatlarını tutan veri modeli.
RayCasting	Kesişim mantığını ve çekirdek matematiği barındırır.
SequentialSolver	Algoritmanın tek thread üzerinde koşan referans sürümü.
ParallelSolver	ForkJoinPool kullanarak işleri dağıtan paralel sürüm.
PolygonUtils	Rastgele nokta üretimi ve poligon inşası araçları.
BenchmarkRunner	Nano saniye hassasiyetinde ölçüm yapan performans modülü.

📊 5. Performans Analizi ve Hızlanma (Speedup)

Testler 8 çekirdekli (Core) bir sistem üzerinde farklı iş parçacığı (Thread) konfigürasyonlarıyla gerçekleştirilmiştir.

Karşılaştırmalı Süre Tablosu (milisaniye)

Test Noktası	Sıralı Süre	2 Thread	4 Thread	8 Thread
100	0.126 ms	0.144 ms	0.105 ms	0.041 ms
500	0.059 ms	0.064 ms	0.056 ms	0.072 ms
1.000	0.097 ms	0.082 ms	0.068 ms	0.090 ms
5.000	0.602 ms	0.369 ms	0.287 ms	0.121 ms
10.000	0.663 ms	0.405 ms	0.579 ms	0.372 ms
50.000	2.298 ms	1.141 ms	0.961 ms	1.066 ms

Hızlanma Katsayısı (Speedup)

Speedup formülü: $Speedup = \frac{T_{\text{sıralı}}}{T_{\text{paralel}}}$

Örneğin **5.000 nokta** için hesaplama: $Speedup = \frac{0.602}{0.121} \approx 4.96x$

[!NOTE] **Analiz Çıktısı:** Tablodan da görülebileceği üzere, düşük nokta sayılarında (100-500) thread oluşturma ve yönetme maliyeti (overhead) avantajı yok edebilmektedir. Ancak nokta sayısı 5.000 ve üzerine çıktığında **~5 katlık net bir performans artışı** elde edilmiştir.

6. Sonuç

Bu çalışmada, Ray Casting algoritması ile PIP probleminin paralel çözümü başarıyla uygulanmıştır.

- Doğruluk:** Sıralı ve paralel uygulamalar hem Convex hem de Concave (Yıldız) poligonlarda %100 eşleşen sonuçlar üretmiştir.
- Performans:** ForkJoinPool kullanımı, özellikle büyük veri kümelerinde donanımın tüm potansiyelini kullanarak hesaplama süresini dramatik şekilde düşürmüştür.
- Ölçeklenebilirlik:** Tasarlanan lock-free mimari sayesinde donanım çekirdeği arttıkça lineer'e yakın bir hızlanma elde edilme potansiyeli kanıtlanmıştır.

Kaynakça

- Skiena, S. S., *The Algorithm Design Manual*, Springer, 2020.
- Cormen, T. H. et al., *Introduction to Algorithms*, MIT Press, 2022.
- Goetz, B., *Java Concurrency in Practice*, Addison-Wesley, 2006.
- O'Rourke, J., *Computational Geometry in C*, Cambridge University Press, 2000.
- Amdahl, G. M., "Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities", AFIPS, 1967.